지능정보시스템 중간고사 정리 (최종)

인천대 컴퓨터공학부 · 2~6주차 · 근거: 수업 슬라이드 원문 + 10/13, 10/15 복습수업 · 시험 10/22(목) 9시, 75분

시험 정보

- 객관식 + 주관식. 주관식은 외우기보다 인과관계(왜 그런지)를 서술. 쓰는 순서: 원인 → 결과 → 결론.
- 코드 빈칸 없음 (구조만 이해). nDCG·cron·bitsandbytes는 출제 안 함. "이 상황에는 무엇을 고르나", "이 증상은 어느 계층 문제이고 어떻게 고치나"를 묻는다.
- 교수님이 해보라고 한 것: ① BPE 손계산 ② documents CRUD를 `메서드 + URI` 표로 설계 ③ 학습 단계별 출력 차이 ④ 증상 → 계층 → 해결 매칭 ⑤ 체인 구성요소와 입출력 형태 맞물림 ⑥ 검색 품질 기법을 상황별로 고르기 ⑦ 청킹 방식·오버랩을 왜 쓰는지/안 쓰는지

이전 버전에서 정정한 것 — ① ANN의 실무 목표 Recall은 0.95~0.99 (이전: 0.90~0.95는 녹음 오인식) ② 하네스의 "환경 설정"은 슬라이드 용어로 "실행 환경"(샌드박스) ③ BM25 TF 상한은 2.5 (= k1+1, k1=1.5) ④ 리랭킹 슬라이드 예시는 상위 30개 → 5개 ⑤ 루프 종료 조건과 안티패턴, 하네스 실패 유형은 슬라이드의 표 그대로 정리

2주차 · LLM 기초와 AI 시스템 개요

LLM · 모델 종류

- LLM = 방대한 텍스트로 학습해 다음 토큰을 확률로 예측하는 모델. 이를 반복해 문장 생성 = 자기회귀. 요약·번역·코드 생성이 모두 "다음 단어 예측"이라 범용. 직접 학습은 불가능 → 가져다 쓰는 법을 배움.

종류	한 줄	생긴 이유
VLM	이미지+텍스트를 같은 임베딩 공간에서 함께 입력	사진·표·화면 이해
MLLM	이미지에 음성·영상까지	다양한 입력
SLM	수억~수십억 파라미터 경량	GPU 부족한 기기에서 실행
LRM	답 전에 추론 과정을 길게 생성(정확, 느리고 비쌈)	초기 LLM에 추론 기능이 없어 추론 과정을 학습
MoE	전체 파라미터는 크지만 입력마다 일부만 활성화	작은 GPU로 좋은 성능

Transformer · Attention

- RNN/LSTM: 순차 처리라 병렬 불가, 긴 문장에서 앞을 잊음 → Transformer(2017): 전체를 한 번에 봄, 핵심 = Attention.
- Q(질문 토큰) K(각 토큰의 표지) V(실제 값). Q·K 내적(유사도) → 나눔(스케일링) → softmax(0~1 비율) → V를 그 비율로 섞음. 멀티헤드 = 여러 관점 동시. 위치 인코딩 = 어텐션은 순서를 모르므로. 요즘은 디코더 전용.

$O(n^2)$: 모든 토큰이 모든 토큰과 비교 → 길이 2배면 계산 4배 → 컨텍스트 윈도우 상한, 길수록 비용·지연 급증 → "다 넣자"가 안 통하고 무엇을 넣을지 고르는 것이 곧 성능이자 비용 (= RAG·컨텍스트 엔지니어링이 필요한 이유).

토큰화 · BPE

- 토큰 = 서브워드. 단어 단위는 신조어·오타 불가, 글자 단위는 너무 길어짐 → 서브워드(BPE)가 절충.
- BPE: 자주 함께 등장하는 문자쌍을 반복 병합, 어휘 크기에 도달하면 중단. 병합마다 어휘 +1. 예) low×5, lower×2, newest×6, widest×3: 초기 글자 10개 → `es`(9회) 병합 → 11 → `est` → 12 …
- 학습 순서: 말뭉치 → 일부로 BPE 학습(어휘·병합 규칙 확정) → 전체 말뭉치를 번호로 변환 → LLM 사전학습. 로컬 모델은 학습에 쓴 토크나이저를 그대로 써야 함.
- 한국어가 토큰을 더 쓰는 이유: 영어 데이터 비중이 커 병합 기회가 적음 + 조사·어미로 같은 어간이 흩어져 빈도 분산("학교에서/를/가") + byte-level BPE에서 한글 1글자 = 3바이트.
- 컨텍스트 윈도우 = 시스템 지시+도구 명세+이력+검색 결과+질문의 합. 요금 = 입력+출력 토큰(출력이 더 비쌈). 대화는 이력을 매번 재전송 → 누적 비용은 턴 수의 제곱에 비례.

학습 3단계 ("대한민국의 수도는")

단계	하는 일	결과
사전학습	대규모 텍스트로 다음 토큰 예측(라벨 없음=자기지도)	이어 쓰기만: "서울이다. 인구는…"
SFT	좋은 질문-답변 쌍 모방. 새 지식이 아니라 응답 형식	"서울입니다."
정렬 (RLHF/DPO)	두 답 중 나은 쪽 학습, 위험 요청 거절	톤 일관, 거절, 모호하면 되묻기

학습 가능성: 사전학습 불가 · 전체 미세조정 사실상 어려움 · LoRA/QLoRA는 GPU 1~2장으로 제한적 · RAG·프롬프트는 학습 불필요.

양자화 · 성능 지표 · 모델 선택

- 양자화 = 숫자 표현 비트를 줄여 메모리 절감(성능을 조금 포기). 손실 이유: 표현 가능한 값이 줄어 서로 다른 값이 같은 값으로 뭉침. 메모리 = 파라미터 수 × 바이트.

형식	값 1개	7B	70B
FP32	4B	28GB	280GB
FP16/BF16	2B	14GB	140GB
INT8	1B	7GB	70GB
INT4	0.5B	3.5GB	35GB

큰 모델 4bit > 작은 모델 16bit(대체로). INT4 미만은 열화 뚜렷. 실제로는 KV 캐시 등이 더 필요(7B INT4 ≈ 5GB).

벤치마크	내용
MMLU	57개 주제 4지선다
GPQA	대학원 수준, 구글링으로 안 풀리게 출제
HumanEval	코드를 실제 실행해 테스트 통과로 판정(pass@k)
SWE-bench	실제 GitHub 이슈+PR 2,294쌍(Verified 500). FAIL→PASS(해결됐나), PASS→PASS(안 망가뜨렸나)

	API 모델	오픈웨이트(로컬)
성능 / 비용	최상위 / 초기 없음, 토큰당 종량	근접·다소 뒤짐 / GPU 서버 필요, 고정비
데이터 · 버전	외부로 전송 · 갱신 시 동작 변경 가능	내부에 머묾 · 버전 고정

AI 시스템 구조 · REST · FastAPI

- 일반 웹 서비스에 오케스트레이션·검색 계층·모델 계층 추가. 프론트엔드 → API 서버(FastAPI) → 오케스트레이션(LangChain) → 임베딩/Vector DB/모델 서빙. 색인 시점(청킹→임베딩→저장) / 질의 시점(질문→검색→프롬프트→LLM→답변). Vector DB에는 벡터+원문+메타데이터. 지연은 대부분 LLM 첫 토큰까지.
- REST = 메서드 + 자원(URI). URL은 전체 주소, URI는 자원 식별 경로. URI는 복수형 명사, 동사는 메서드 (`POST /createUser` ✕ → `POST /users` , `DELETE /users/1` ◯).

메서드	의미	안전	멱등	documents CRUD	응답
GET	조회	O	O	`GET /documents` , `GET /documents/{id}`	200 / 404
POST	생성	X	X	`POST /documents`	201
PUT	전체 교체	X	O		
PATCH	부분 수정	X	X	`PATCH /documents/{id}`	200
DELETE	삭제	X	O	`DELETE /documents/{id}`	204

상태 코드: 2xx 성공(200, 201, 204) · 4xx 클라이언트 잘못(400, 401, 403, 404, 422, 429) · 5xx 서버 잘못(500).

- FastAPI 3요소: Starlette(라우터: 메서드+URI로 함수 호출, ASGI) · Pydantic(검증·문서) · Uvicorn(요청 수신). WSGI(Flask)는 요청 끝날 때까지 워커 점유, ASGI는 대기 중 다른 요청 처리.

> 비동기가 중요한 이유: LLM 호출은 계산(ms)이 아니라 네트워크 대기(2~10초)가 대부분 → 동기면 워커가 묶여 처리량 급감. 처리량 = 워커 수 ÷ 평균 응답 시간(4 ÷ 5초 = 0.8건/초). 비동기는 `await`에서 대기 목록에 넣고 다른 요청 처리 → 워커 1개로 수백 건 대기. 워커 늘리기는 워커당 메모리 수백 MB라 비현실적.

- Pydantic: 틀린 요청은 엔드포인트 진입 전 422 차단. `max_length` 없으면 10만 자 → 토큰 비용 사고, `top_k` 상한, 모델명 `Literal`, 응답 스키마 고정. 구조화 출력 = 미리 정한 형식(JSON)으로 출력, 코드가 바로 쓸 때 필요, 형식을 어길 수 있어 검증+재시도. 스트리밍 = 8초 빈 화면 대신 0.8초에 첫 글자(TTFT 첫 토큰까지, TPS 초당 토큰).

3주차 · 프롬프트 · 컨텍스트 엔지니어링

엔지니어링이 필요한 3가지 배경

배경	내용 → 결론
① 모델을 학습시킬 수 없음	사전학습 불가·미세조정 제한 → 모델 주변(프롬프트·컨텍스트)을 설계. 가장 쉬운 프롬프트부터
② 출력이 확률적	같은 입력도 매번 다름(temperature 0이어도 완전히 같진 않음). 통과/실패 대신 지표(근거 충실·질문에 답함·형식·출처)로 측정 → 출력 형식 강제·검증, 실패 시 재시도 경로, 품질 반복 측정
③ 컨텍스트가 유한하고 비쌈	합산 한도, 길수록 비용·지연 ↑ ($O(n^2)$), 많이 넣는다고 좋아지지 않음(불필요한 정보는 간섭, 가운데 정보는 반영 약함) → 무엇을 넣을지 고르는 것이 성능이자 비용

순서: 프롬프트 → 컨텍스트 → 하네스 → 루프 (뒤가 앞을 포함). "요령(문구 복사)"에서 "공학(실패 유형 분류·버전 관리·지표 측정·계층별 책임 분리)"으로. 같은 모델도 입력(역할·규정·형식·근거)에 따라 결과가 완전히 다름.

프롬프트 · 컨텍스트

- 프롬프트 엔지니어링 = 질문을 잘 하는 것(시스템 프롬프트, 사고 유도 CoT, 구조화 출력). 구조화 출력은 코드가 출력을 바로 쓸 때(에이전트 도구 호출 명령).
- 컨텍스트 엔지니어링 = 모델이 한 번 호출할 때 보는 것 전부 설계: 시스템 프롬프트+대화 이력+검색 결과+도구 명세+사용자 질문.
- Lost in the middle: 정답 문서를 앞/뒤에 두면 잘 쓰고 가운데면 성능 하락(문서 없이 답할 때 56.1%보다도 낮아지기도). → 중요한 건 앞뒤.

4전략	핵심
선별	질문과 관련된 것만(= RAG, 상위 k)
압축	길어진 이력 줄이기
격리	서브에이전트로 역할 분담(컨텍스트 창이 여러 개). 단점: 수행이 오래 걸림
순서	가운데는 놓치므로 중요한 건 앞뒤

이력 압축	방법	장점	단점
전체 유지	전부 보관	단순·손실 없음	토큰 폭증
슬라이딩 윈도우	최근 N턴만	토큰 적음	이전 대화를 완전히 잊음
요약	최근 N턴+과거는 LLM 요약 1개	토큰 ↓ 맥락 일부 유지	세부 손실, 요약 비용
선별 검색	전부 저장, 지금 질문과 관련된 과거만 검색	필요한 기억만	벡터 DB 필요, 검색 성능 의존

4주차 · 하네스 · 루프 엔지니어링

하네스 = 모델을 실제로 동작시키는 바깥 껍데기 (가중치를 제외한 모든 것)

도구 정의·권한·실행 환경·컨텍스트 조립·검증·로깅. 같은 모델도 하네스가 다르면 결과가 다름 → 모델 교체가 아니라 하네스 개선이 답인 경우가 많음.

구성요소	내용
도구 레지스트리	도구 이름·설명·입력 스키마·반환 형식. 설명이 곧 프롬프트(모델은 설명만 보고 선택). 흔한 실수: 도구 하나가 너무 많은 기능 / 기능이 겹치는 도구 / 반환값이 장황해 컨텍스트 낭비. 도구 수가 늘면 명세만으로 컨텍스트 잠식
권한·승인	조회는 자동 허용, 변경은 승인, 되돌릴 수 있는 작업과 없는 작업 구분. 권한 최소화. 자연어 지시로 권한을 대신하지 말 것(코드 수준에서 막아야)
실행 환경	모델이 만든 코드가 실제로 돌아가는 자리: 파일 시스템 범위·네트워크·실행 시간·자원 상한. 격리(샌드박스): 컨테이너, 임시 디렉터리
컨텍스트 조립	컨텍스트 엔지니어링의 구성요소를 모아 조립
검증 게이트	출력을 그대로 믿지 않고 통과 조건: 형식(스키마) / 내용(금지 항목) / 동작(실행 통과). 없으면 잘못된 결과가 전파·누적. 하네스에서 가장 자주 빠지는 요소
상태·관측	상태 = 세션을 넘어 남는 것(진행 기록 파일, 작업 목록, 중간 산출물; 컨텍스트는 사라져도 파일은 남음). 관측 = 무슨 일이 있었는지(어떤 도구를 어떤 인자로, 몇 번 반복, 어디서 실패, 토큰·비용). 관측 없으면 개선 불가

> 도구 호출 동작(슬라이드 4단계) "오늘 서울 날씨 알려줘": ① 도구 레지스트리를 JSON 스키마로 LLM 컨텍스트에 주입 → ② LLM이 요청과 도구 설명을 비교해 `get_weather` , 파라미터 city=Seoul 판단 → ③ 구조화된 호출 텍스트(JSON/전용 태그) 생성 — LLM은 직접 코드·API를 호출할 수 없음 → ④ 하네스가 출력을 감지·파싱해 도구 레지스트리에서 실제 코드를 찾아 실행하고 결과를 LLM에 전달.
> 도구가 수십~수백 개면: 벡터 검색(도구 설명을 임베딩, 질문과 가까운 상위 N개만) / 라우터 모델(경량 분류 모델이 의도 파악 → 도구 세트 선택) → 선별된 3~5개 스키마만 LLM에 전달.

증상	실제 원인	고칠 곳
엉뚱한 도구를 호출	도구 설명이 모호하거나 기능이 겹침	도구 레지스트리
위험한 작업을 그냥 실행	자연어 지시에만 의존	권한과 승인
잘못된 결과가 계속 전파	통과 조건이 없음	검증 게이트
세션마다 처음부터 다시	상태를 파일로 남기지 않음	상태 저장
어디서 틀렸는지 모름	호출 기록이 없음	관측·로깅
도구 명세만으로 창이 참	도구가 너무 많음	도구 선별

모델 교체가 답	하네스 개선이 답
추론 능력 자체 부족 · 긴 문맥을 못 다룸 · 특정 언어 성능 낮음 · 도구 호출 형식을 자주 어김	도구를 잘못 고름 · 같은 실수 반복 · 중간에 목표를 잃음 · 결과를 검증하지 않고 끝냄

- 설계 원칙: ① 가장 단순한 구성에서 시작(단일 호출로 되는 일을 에이전트로 만들지 않음) ② 모든 구성요소는 "모델이 이건 못 할 것"이라는 가정을 담음 → 모델이 좋아지면 만료, 주기적으로 걷어냄 ③ 사람이 판단할 지점을 남김.
- 추가 에이전트: 라우터(의도 분석→서브 에이전트/도구 세트) · 플래너(복잡한 목표를 단계별 하위 작업으로) · 검증/평가(2차 검증, 검증 게이트 역할) · 가드레일(프롬프트 주입 입력, 개인정보 유출 출력 감시 = 방화벽, 입출력 모두). 분리 이점: 소형·저렴한 모델로 라우팅·보안 검사, 고난도만 플래그십 → 비용·속도 / 에이전트마다 도구 스키마 2~3개+전용 프롬프트 → 지침 준수율↑.
- 장기 실행 하네스: 작업이 컨텍스트 창을 넘으면 세션이 바뀔 때 기억 상실 → 초기화 에이전트(환경·작업 목록을 한 번 생성) + 작업 에이전트(세션마다 조금씩). 이어 주는 것은 컨텍스트가 아니라 파일(진행 기록·기능 목록·커밋 로그). 한 번에 하나씩(한 세션 한 기능, 테스트 통과 후 커밋).

루프 엔지니어링

- 필요성: 오류 누적(단계별 90%여도 10단계면 ≈35%), 모델은 자기 출력이 틀렸는지 알려주지 않음 → 중간 확인·수정·재시도. 생성은 싸고 검증은 비쌈 → 검증이 병목.
- 기본 루프: 관찰(상태 확인) → 행동(모델 호출·도구 실행) → 검증(목표 만족 판정, 통과 못 하면 채택 안 함) → 복구(실패 원인을 반영해 재시도). 같은 입력 재시도 = 같은 실패 → 무엇이 틀렸는지를 입력에 넣어야.
- 검증 설계: 검증기를 먼저 만들 것. 좋은 검증 = 기계가 판정 / 이분법 / 모델이 조작 불가. 모델이 자기 결과를 판정하면 같은 컨텍스트라 같은 실수 → 판정용 모델은 별도 컨텍스트.

종료 조건	조건	주의
성공 종료	검증 통과	가장 바람직
횟수 상한	N회 반복 후 중단	N을 너무 크게 잡지 말 것
예산 상한	토큰·시간 한도	비용 사고 막는 최후 방어선
정체 감지	진전 없으면 중단	같은 실패 반복 탐지
사람 호출	판단이 필요하면 넘김	되돌릴 수 없는 작업 앞

안티패턴	무슨 일	대응
무한 재시도	같은 실패 반복	횟수·예산 상한
목표 없는 반복	완료 정의 없음	검증 조건 먼저 정의
검증 없는 루프	틀린 결과가 쌓임	판정 단계 삽입
실패 원인 미전달	같은 입력→같은 실패	오류 내용을 다음 입력에
조기 완료 선언	미완성인데 끝났다고 보고(self-approving)	분리된 평가 에이전트(검증기)

- 하네스와 루프의 경계: 하네스 = 한 번의 실행을 가능하게 하는 환경(도구·권한·검증 수단·로깅), 루프 = 그 실행을 언제 다시 돌릴지(트리거·목표·검증·종료 조건). 하네스 먼저, 루프는 그 위(하네스가 부실하면 루프가 약점을 증폭).
- 필요한 경우: 한 번 묻고 답하면 끝 → 프롬프트·컨텍스트(챗봇·번역기·요약기·사내 검색) / 아니면 하네스·루프(코딩 에이전트: 작성→테스트→실패→수정, 리서치, 컴퓨터 사용, 환불 처리처럼 실행이 들어가는 업무 자동화, 데이터 분석).

5주차 · RAG · 임베딩 · 청킹 · 검색 품질 · 벡터 DB

RAG 개요

- RAG = 질의 시점에 관련 문서를 검색해 근거로 넣고 생성. 모델은 그대로 입력을 바꿈, 지식은 모델 밖(DB)에 둠 → 갱신 시 색인만.
- 색인 시점: 수집 → 파싱(PDF 표·스캔·한글 파일) → 청킹(너무 크면 무관 내용 섞임, 너무 작으면 문맥 끊김) → 임베딩(조각마다 1회, 100만 조각=100만 호출) → 저장(벡터+원문+메타데이터).
- 질의 시점: 질문 임베딩(색인과 같은 모델) → 유사도 검색(k는 보통 3~10, 권한 필터를 여기서) → 프롬프트 조립(지시+근거+질문) → 생성("근거에 없으면 모른다고 답하라" 필수 = 환각 1차 방어) → 응답(출처 함께).
- 장점: 모르던 지식(사내 규정·최신) · 환각 감소(없어지진 않음) · 출처 제시 · 갱신 용이(그 조각만 재색인) · 비용 효율. 단점: 검색 실패하면 끝(RAG 품질 = 검색 품질) · 문서 가로지르는 질문 약함 · 근거가 있어도 무시(Lost in the Middle) · 근거 자체가 틀림(오래됨·모순).

RAG의 종류

기본(Naive) RAG = 청킹→임베딩→상위 k→생성. 한계: 검색 1회뿐 / 조각 간 관계 없음 / 질문 난이도와 무관한 같은 경로 / 텍스트만 → 변형이 생김. 무엇을 바꾸는가로 3갈래: 색인·검색 개선 / 색인 구조(GraphRAG) / 판단·검증·반복(Self·CRAG·Adaptive·Agentic) (+입력 확장 Multimodal).

색인·검색 개선	동작	극복 한계
하이브리드	벡터+키워드(BM25)	고유명사·코드·숫자
Reranking	상위 30개를 정밀 모델로 재정렬해 5개	1차 검색 순위 오류
HyDE	가상 답변을 생성해 그걸로 검색	질문-문서 표현 차이
질의 재작성·Fusion	질문을 여러 형태로 바꿔 검색 후 합침	질문 하나에 의존
Contextual Retrieval	조각마다 문서 맥락 요약을 앞에 붙여 색인	조각이 문맥을 잃음
RAPTOR	조각을 재귀 요약해 트리로 색인	상위 개념 질문에 조각이 너무 세밀

종류	핵심	강한 질문 / 약점·비용
GraphRAG	질문→엔티티 top k→그래프(관계·이웃·조각·보고서)→생성	관계·전체 조망("A 사건과 B 규정의 연결"). 색인 비용 매우 높음(LLM으로 모든 문서를 훑어 엔티티·관계 추출), 그래프 품질=추출 품질, 단순 사실엔 이득 없음. 대부분은 기본 RAG로 시작
Self-RAG	생성 모델 자체가 ①검색 필요? ②조각 관련? ③답이 근거로 뒷받침? 판단하도록 학습(reflection token 미세조정), 별도 평가기 없음	불필요한 검색 건너뜀·근거 없는 답 스스로 걸러냄 / 학습 필요 → API 모델엔 바로 적용 불가
CRAG	검색 결과를 판단하는 별도 평가기(모델)가 쓸지 말지 결정	근거 신뢰가 중요한 곳. Self-RAG와 차이 = 판단 주체가 따로
Adaptive	질문 복잡도로 경로 분기: 간단(검색 없이) / 보통(검색 1회) / 복잡(여러 번 검색·추론). 분류는 소형 분류 모델·LLM(라우팅 패턴)	쉬운 질문에 비싼 경로 안 씀 → 비용·지연 절감
Agentic	에이전트를 검색 루프에(결과 나쁘면 재검색, 종료 조건까지)	다단계 추론·리서치 / 호출 수만큼 비용 증가
Multimodal	표·차트·도면·스캔·이미지를 검색 대상에. ①이미지를 텍스트 설명으로 색인 ②같은 공간에 임베딩(CLIP)	표·이미지가 핵심인 문서

위로 갈수록 단순하고 쌈. 대부분 기본 + 색인·검색 개선으로 충분, 나머지는 필요가 확인될 때만.

임베딩

- 텍스트 → 고정 길이 실수 벡터, 의미가 비슷하면 가깝다. 단어 임베딩(2013, Word2Vec) → 문장 임베딩(2019, Sentence-BERT; RAG는 조각 단위라 필요, 단어 벡터 평균은 "이월할 수 있다/없다"를 구분 못 함). 학습 = 대조 학습(비슷한 쌍은 가깝게, 다른 쌍은 멀게).

- 유사도: 코사인(방향, −1~1, 텍스트 검색 기본) / 내적(정규화 벡터에서 코사인과 동일, 가장 빠름 — 실무는 정규화+내적) / 유클리드(거리, 낮을수록 유사).
- 모델 종류: 상용 API(고품질·과금·외부 전송) / 오픈 다국어(BGE-M3, multilingual-e5) / 한국어 특화(KoSimCSE, ko-sroberta; 혼용 문서에 약할 수 있음) / 멀티모달(CLIP).
- 선택 기준: 차원(384·768·1024·3072; 크면 표현력↑ 저장·검색 비용↑) · 언어(영어 전용에 한국어 → 품질↓) · 최대 입력 길이(512 흔함, 넘으면 잘림 → 청킹 크기와 맞춤) · 비용·속도 · 한 번 정하면 바꾸기 어려움(전체 재색인). 저장 용량 = 조각 수 × 차원 × 4B(100만×1024×4 ≈ 4GB).
- 한계: 고유명사·코드·숫자(`ERR-4021` ≈ `ERR-4012`) · 부정·미묘한 차이("이월 가능/불가") · 도메인 어휘(사내 약어) · 길이 불균형(짧은 질문↔긴 조각 → HyDE). 임베딩만으로 검색이 완성되지 않음.
- 색인·질의의 임베딩 모델은 반드시 같아야. 임베딩 모델과 생성 LLM은 독립(LLM에는 원문 텍스트가 전달). LLM 교체=설정 변경, 임베딩 교체=전체 재색인.

청킹

- 자르는 이유: 컨텍스트 한도 · 임베딩 한도(512~8192) · 정보 희석. 자르면 생기는 문제: 맥락 단절 / 대명사("이 경우") / 경계(답이 두 조각에 걸침). = 검색될 만큼 작게, 뜻이 통할 만큼 크게. 같은 문서·모델이라도 청킹만 바꿔 재현율이 달라짐(과제: 500자 0.65 → 조 단위 0.85).

방식	방법 · 특징
고정 길이	단어·문장 무관 N자마다. 가장 쉬움, 문장이 잘림 → 오버랩 필수
재귀 분할	구분자 문단→줄→단어→글자, 청크 크기 100자: 문단 A 60 OK / B 250 큼→줄로(B1 90 OK, B2 160 큼→단어로) / C 40 OK / 올라가며 합치기 A(60)+C(40)=100. 기본값인 이유: 구조 대략 존중·크기 보장·싸고 빠름(문자열 처리만)·형식 몰라도 동작
구조 기반	제목·조항·표·함수 등 문서 구조를 따라(과제의 `## 제N조`). 재귀는 1순위 기준 크기·경계는 모양, 구조 기반은 구조·경계는 의미, 크기 제각각, 구조가 파싱돼 있어야 함. 절차: ①구조 파싱 ②섹션별 분리 ③경로 부착(제12조 ② 같은 소속을 머리말·메타데이터) ④크기 보정. → 구조로 크게, 넘치는 부분만 재귀
의미 기반	문장 분리 → 각 문장 임베딩 → 이웃 문장 벡터 거리가 크게 튀는 지점이 경계(앞뒤 1~2문장 묶으면 덜 흔들림). 장점: 한 주제만 담겨 벡터 선명. 단점: 길이 가변, 문장 수만큼 임베딩 호출(비용↑), 임계값을 문서마다 조정, 표·코드엔 불가. 구조 없는 긴 산문(회의록·인터뷰)

- 크기: 너무 작으면(수십 토큰) 검색은 잘 맞지만 문맥 없음(k 키우면 토큰·잡음↑) / 너무 크면(수천) 여러 주제 섞여 벡터 흐림·순위 하락·컨텍스트 차지. 시작점 256~512 토큰 + 오버랩 10~25%. 짧은 문서(FAQ·공지)는 문서 하나=조각 하나.
- 증상별 대응: 정답이 상위 k에 없음 → 조각을 더 작게·오버랩↑ / 정답은 있는데 순위 낮음 → 맥락 붙임·메타데이터 필터 / 관련 없는 조각이 올라옴 → 조각을 더 크게·구조·의미 기반.
- 오버랩 = 이웃 조각 경계를 겹침(경계 문제의 보험: 경계 근처 문장이 최소 한 조각엔 온전히). 보폭 = 크기 − 오버랩(500/100 → 0~500, 400~900, 800~1300). 대가: 조각 수↑(10만 토큰·500 기준: 오버랩 0→200개, 50→약 223(+11%), 125→약 267(+33%), 250→400(+100%)). 부작용: 이웃 조각이 k 자리를 중복 점유(대응: 인접 병합·중복 제거·MMR). 한계: 바로 옆만 보충(멀리는 맥락 머리말·Parent-Child). 방식별: 고정=필수 / 재귀=문장·단어 경계에 맞춰 이내 / 구조 기반=조항 경계 불필요, 긴 섹션 내부만.

문서 유형	권장	문서 유형	권장
규정·약관	조·항 단위(질문 단위와 일치)	코드·설정	함수·블록 단위
매뉴얼·가이드	제목 단위	표·시트	표 단위+머리글 반복
FAQ·Q&A	질문 하나당 한 조각	긴 계약서	조항+맥락 머리말
회의록·보고서	고정 크기+오버랩		

검색 품질

- RAG 실패의 대부분은 검색 실패 → 정답 조각이 프롬프트에 들어갔는지 먼저 확인한 뒤 프롬프트·모델을 건드린다. 실패 유형: 못 찾음(후보에 없음 → 청킹·하이브리드·질의 재작성) / 잘못 고름(후보엔 있는데 상위 k 밖 → 리랭킹·k 조정) / 안 씀(프롬프트엔 있는데 반영 안 됨 → 위치·지시문·컨텍스트 예산).
- 벡터만으로 부족: `KCC-3` ≈ `KCC-5`, 사람 이름·부서명·함수명, 학습 분포 밖 단어(사내 약어·제품 코드·신조어), 짧은 질의 / 반대로 어휘 검색도 못 하는 것: "휴학 신청"으로 "휴학계 제출"을 못 찾음.

- BM25 = (단어가 얼마나 드문가 IDF: 그 단어를 포함한 조각 수 기준, 드물수록↑) × (조각에서 얼마나 자주 TF: 많을수록↑ 하지만 상한 2.5) ÷ 길이 보정(긴 조각은 우연히 포함될 확률↑ → 분모; `b=0` 무시, `b=1` 완전 비례, 0.75 관례). 뜻 대신 글자 일치.

	밀집(Dense)	희소(BM25)
비교 대상	의미 벡터	단어 일치
강한 / 약한	다른 표현·같은 뜻 / 낯선 고유명사	고유명사·코드·약어 / 동의어·의역
색인·새 문서·모델 교체	임베딩 계산 / 계산 후 반영 / 전체 재임베딩	역색인, 가벼움 / 즉시 반영 / 영향 없음
설명 가능성	어려움	어느 단어가 맞았는지 보임

> 하이브리드 + RRF: 질문 → 벡터 검색 상위 50 + BM25 상위 50 → RRF로 합침. 코사인(0~1)과 BM25(상한 없음)는 단위가 달라 점수는 버리고 순위만. RRF = Σ 1/(k+순위), k=60. 예) A: 벡터 1위·BM25 2위 → 1/61+1/62 ≈ 0.0325. 순위가 높을수록 점수↑, 양쪽 모두 상위권이 최고. 양쪽 1위 = 2/61 ≈ 0.0328.

- 리랭킹(2단계): 1차(벡터+BM25, 상위 30) → 교차 인코더로 재정렬 → 상위 5. 바이 인코더(질문·조각을 따로 벡터화, 코사인; 빠름·조각 벡터 사전 계산)와 달리 교차 인코더는 질문+조각을 한 입력으로 붙여 Transformer 어텐션으로 점수 하나(스칼라) → 정밀하지만 후보마다 모델을 돌려 느림(그래서 상위 몇십 개에만) → 기다림을 허용하는 시스템에서.
- 질의 재작성: 기본 재작성("그거 언제야?"→"2026학년도 2학기 수강신청 시작일은?") / 다중 질의(여러 관점으로 늘려 각각 검색→RRF) / HyDE(가상 답변으로 검색; 짧은 질문↔긴 문서 괴리 완화, 답변이 틀려도 형식은 비슷).

상황	쓸 것	비고
대화형 챗봇	질의 재작성	대명사·생략 많아 사실상 필수
질문이 다의적	다중 질의	관점별 검색 후 결합
질문체와 문서체가 다름	HyDE	규정·논문처럼 문체가 고정된 문서에 효과
고유명사·코드 질의	하이브리드	재작성보다 어휘 검색이 먼저
지연이 중요한 서비스	재작성 생략	LLM 호출이 늘어서

지표	측정	지표	측정
Recall@k	정답이 상위 k 안에 들어왔는가(과제)	MRR	첫 정답 순위의 역수(1위=1, 3위=1/3) 평균, 정답 하나인 질의에 적합
Precision@k	상위 k 중 관련 있는 비율(5개 중 3개=3/5)	Hit Rate	정답이 하나라도 들어왔는가(가장 단순)

평가 데이터셋 = 질문+정답 조각 쌍: 실제 질문 로그(서비스 이후) / 문서에서 LLM 역생성 / 담당자 직접 작성. 50~100문항이면 충분, 파라미터를 바꿀 때마다 같은 질문 집합으로 재측정(개선 여부 확인의 유일한 방법).

개선 우선순위	조치	이유
1	정답 문서가 색인에 들어 있는지	없으면 뒤 작업 무의미
2	파싱 결과 육안 표본 점검	깨진 텍스트는 어떤 검색으로도 못 살림
3	청킹 단위를 문서 구조에 맞춤	재현율이 가장 크게 움직이는 구간
4	하이브리드 도입	고유명사·코드 질의 즉시 개선
5	리랭킹	상위 순서 정리, 지연 비용
6	질의 재작성·다중 질의	대화형에서 효과, LLM 호출 증가
7	프롬프트·모델 조정	위 단계를 마친 뒤에

벡터 DB와 인덱스

- 정확 탐색(Flat): 모든 벡터와 비교, 항상 정확(recall 1.0), 삽입·삭제 자유. 비용 = 조각 수 × 차원(선형). GPU 행렬 곱이면 100만 개도 수십 ms. DB의 풀스캔과 같은 원리. 수만 개 이하는 인덱스 불필요(수십만도 요청이 많지 않으면 정확 탐색 가능, 100만+부터 인덱스). 정확 탐색 시작의 이점: recall 1.0이라 인덱스 탓/임베딩 탓 분리, 나중에 ground truth. 인덱스는 성능 문제가 측정으로 확인된 뒤에.
- B+Tree를 못 쓰는 이유: 정렬 가능한 키를 트리로 만드는 구조인데 고차원 벡터는 정렬 불가("어느 쪽으로 갈지" 정할 수 없음).

	B+Tree	벡터 인덱스(ANN)
대상 / 질의	정렬 가능한 키 / =, <, >, 범위	고차원 실수 벡터 / 가장 가까운 k개
정확도 / 비용	항상 정확 / log n 보장	근사(놓칠 수 있음) / 파라미터로 조정
갱신 / 메모리	삽입·삭제 자유 / 디스크 친화	구조에 따라 재구축 / 대부분 메모리 상주

- ANN과 Recall@k = 정확 탐색 상위 k 중 인덱스가 찾은 비율(10개 중 9개 → 0.9). 1.0을 포기하는 이유: 마지막 5%에 속도가 수십 배 느려짐 / RAG는 k개 중 하나만 정답이면 충분한 경우 많음 / 리랭킹이 누락을 흡수. 실무 목표 Recall 0.95~0.99(자기 데이터로 측정).
- 3전략: 구역 나누기(IVF, LSH) / 이웃 잇기(HNSW, NSG, DiskANN) / 압축(PQ, 스칼라·이진 양자화). 앞의 둘은 저장 구조, 압축은 비교 비용 → 함께(IVF-PQ, HNSW-PQ). `IVFFlat` =압축 없는 IVF.

	IVF (Inverted File)	HNSW
원리	k-means로 nlist개 구역, 각 벡터를 가까운 구역에 배정, 중심 저장. 검색: 질문↔중심들 비교 → 가까운 nprobe개 구역만 비교. 예) 100만, nlist=1,000, nprobe=10 → 1,000+10×1,000 = 약 11,000회 (1%)	벡터를 이웃과 잇는 그래프, 층 여러 개(위=성기게, 아래=촘촘; 위층 노드는 아래층에도 존재, 위층 진입 확률 낮음). 검색: 최상층 진입 → 층 안에서 더 가까운 이웃이 없을 때까지 이동 → 한 층 내려감 → 반복 → 최하층에서 top-k. 경계 문제가 없어 recall 높음, 탐색이 log n에 가까움
파라미터	nlist(구역 수, 색인 시): ↑ 구역이 작아 빠르지만 경계가 늘어 놓칠 확률 ↑; 시작 = 조각 수의 제곱근 (100만→1,000). nprobe(탐색 구역 수, 질의 시): ↑ recall ↑ 속도 ↓, 시작 = nlist의 1~5%, 질의마다 변경 가능. pgvector: lists/probes	M(노드당 최대 연결, 보통 12~48·기본 16): ↑ 촘촘→recall ↑ 메모리 ↑ 구축 ↑. ef_construction(구축 시 후보 폭, 64~200): ↑ 연결 품질 ↑ 구축 느림(지역 최적해 방지, 재구축 전까지 고정). ef_search(검색 시 후보 수): ↑ recall ↑ 속도 ↓, 질의마다 조정. 조정 순서: M·ef_construction 기본값 → ef_search로 recall → 부족하면 M ↑ 재구축
약점	k-means라 데이터가 어느 정도 쌓인 뒤 구축, 분포가 크게 바뀌면 재구축, 경계 문제	메모리 큼, 구축 느림, 삭제 어려움(삭제 시 그래프가 끊겨 외딴 노드 → 삭제 표시만 하고 주기적 재구축, 갱신=삭제+삽입). 삽입 = 층을 확률로 정하고 이웃 M개 연결(검색과 같은 과정+연결). 빈 상태에서 시작 가능
비교	recall 다소 낮음 · 메모리 적음 · 구축 빠름(데이터 필요) · 메모리 제약·대규모에 적합	recall 높음 · 메모리 많음 · 구축 느림(빈 상태 가능) · 정밀도 우선, 일반적 선택

> PQ (Product Quantization) — 벡터를 잘라 각 조각을 코드북 번호로. ① 1,024차원을 8부분×128차원으로 ② 부분마다 k-means(k=256)로 코드북(대표 벡터 256개, 부분마다 독립) ③ 각 부분을 가장 가까운 대표 번호로 치환 `[17, 203, 88, 5, 141, 250, 66, 32]`. 256인 이유 = 번호가 1바이트(2^8). 4,096B → 8B, 약 512배 압축(Product=조합, $256^8 \approx 1.8\times10^{19}$).
> 검색: 질문도 8부분으로 쪼개 코드북 256개와 거리 → 거리표(8×256=2,048회)를 질문당 한 번 → 저장 벡터는 번호로 표를 조회해 더하기만(곱셈 1,024번 → 표 조회 8번+덧셈; ADC). 대가: 정밀도 손실(같은 대표로 뭉쳐 순위 뒤바뀜) → 재정렬(re-scoring): PQ로 후보 1,000개 → 원본 벡터로 재계산 → 상위 10개. 메모리 부족한 수억 개 대규모에서 IVF와 결합(IVF-PQ).

- 벡터 DB = 인덱스+저장소+질의 기능을 묶은 시스템(HNSW 등 인덱스는 라이브러리). 저장: 벡터+원문 조각(답변 근거)+메타데이터(문서 ID·출처·페이지·부서·갱신일). 메타데이터 이유: 권한 필터·출처 표시·바뀐 조각만 재색인. RDB 대비: 벡터 타입(RDB는 없음)·유사도 검색(`ORDER BY 거리 LIMIT k`)·ANN 인덱스(B+Tree 무용)·필터+벡터 결합·대규모 저장(압축·분산).
- pgvector = PostgreSQL 확장(`CREATE EXTENSION vector`): HNSW·IVFFlat 제공, 연산자 `<->` 유클리드, `<=>` 코사인 거리(=1−코사인 유사도, 작을수록 비슷), `<#>` 음의 내적. 장점: 운영·백업·트랜잭션 그대로, JOIN 가능. 한계: 수천만 건 이상 제한, 차원 상한.

구분	대표	특징
경량·임베디드	Chroma, LanceDB	설치 간단, 파이썬 안, 프로토타입·과제
오픈소스 서버	Qdrant, Milvus, Weaviate	필터 인식 검색, 분산, 대규모, 자체 운영
관리형	Pinecone 등	운영 부담 없음, 종량 과금, 데이터 외부
RDB 확장	pgvector 등	기존 스택 유지, 중소 규모 실용

pgvector로 충분	전용 DB 검토
조각 수백만 이하 · 이미 PostgreSQL 사용 · 다른 표와 JOIN 필요 · 운영 인력 적음	수천만 이상·분산 필요 · 필터가 복잡·빡빡 · 초당 수백 질의 이상 · 벡터가 주 데이터인 서비스

- 필터(권한: 인사팀은 인사팀 문서만): 후처리 필터 = 먼저 검색(상위 5)하고 메타데이터로 거름 → 조건에 맞는 것만 남아 k개를 못 채움(1개만 남을 수 있음) / 필터 인식 검색 = 탐색 중에 조건을 걸어 항상 k개(Qdrant·Milvus). RAG는 권한 필터를 항상 쓰므로 DB 선택 시 확인.
- 운영: 문서가 바뀌면 그 문서 조각을 삭제 후 재삽입(`doc_id`로 묶기) / 임베딩 모델 교체=전체 재색인(차원 바뀌면 테이블도) / 인덱스는 대량 삽입 후 구축이 빠름 / HNSW는 그래프가 메모리에 있어야 빠름(RAM 산정) / recall·지연을 자기 데이터로 주기 측정(정확 탐색 결과 보관) / 벡터 컬럼 차원=임베딩 모델 차원, 색인·검색 함수는 같은 모델.

6주차 · LangChain (LangGraph는 LangChain까지만 다룸)

> 교수님 힌트: 코드를 외우지 말고 구성요소를 체인으로 엮는 방식을 이해. 특히 앞 구성요소의 출력 형태가 다음 구성요소의 입력 형태와 맞물려야 체인이 된다. 구성요소들이 주어지면 "어디가 어긋나는가 / 사이에 무엇이 필요한가"를 물을 수 있음.

- 직접 짠 RAG의 문제: 모델 교체·스트리밍·병렬화·재시도·폴백·로깅·배치가 본래 기능이 아닌데 본체 코드를 건드림(스트리밍=호출부·반환 타입·모든 호출 지점 / 병렬화=함수를 비동기로 바꿔 전 경로 전파 / 재시도=호출마다 예외 처리 중복 / 폴백=호출마다 분기문 / 로깅=단계마다 기록 코드 / 배치=반복문 새로 작성).
- 해법: 공통 인터페이스 = Runnable("입력 하나 받아 출력 하나"). 같은 모양이면 이어 붙이기·감싸기(재시도·로깅)·바꿔 끼우기. 제어의 방향이 뒤집힘(라이브러리=내 코드가 호출 / 프레임워크=프레임워크가 내 코드를 호출). 대가: 모양이 안 맞는 요구가 생기면 불편.
- LangChain의 해결 4문제: 입출력 다루기 / 여러 단계 잇기 / 기억 유지 / 바깥과 연결. 특징 3가지 = 추상화·표준화·체이닝.

구성 요소	하는 일	RAG 단계
Document	텍스트+메타데이터 한 덩어리	
DocumentLoader	원본에서 읽어 옴	수집
TextSplitter	조각으로 자름	청킹
Embeddings	벡터로 바꿈	임베딩
VectorStore	저장하고 찾음	저장·검색
Retriever	질문을 받아 조각 목록 반환	검색(질의)
PromptTemplate	프롬프트 조립	프롬프트
(Chat)Model / OutputParser	생성 / 출력 형식 검증	생성

- 거의 모든 것이 Runnable(프롬프트·모델·파서·검색기·일반 함수; 내 함수는 `RunnableLambda`로 감싸면 체인에). 합성한 결과도 다시 Runnable(연산이 닫혀 있음).
- LCEL: 파이프 `|`로 연결 `chain = prompt | model | output parser`. 프롬프트 템플릿(입력을 모델이 읽을 형태로, 중괄호 자리에 값) / 모델(호출, temperature 낮을수록 덜 흔들림) / 출력 파서(`StrOutputParser`: 응답 객체에서 문자열만). 체인을 만드는 줄에서는 아직 아무것도 실행되지 않음. `invoke` 입력은 딕셔너리, 키 이름이 템플릿의 중괄호 이름과 같아야 함.

메서드	하는 일	쓰는 상황
`invoke`	한 건, 완성된 결과를 한 번에(기다림)	가장 기본
`stream`	청크 단위 실시간 출력, 첫 글자가 빨리 뜸. 최종 결과는 invoke와 같음	장문 응답·사용자 화면
`batch`	여러 입력 일괄, 입력과 같은 순서의 리스트, 내부 병렬, `max_concurrency`로 동시 수 제한(API 속도 제한)	평가 질문 집합·대량 처리
`ainvoke / astream / abatch`	위 셋의 비동기(`async/await` 환경에서만)	웹 서버(FastAPI) — 2주차 비동기와 같은 이유
`RunnableParallel`	여러 체인 동시 실행, 이름이 결과 딕셔너리의 키, 같은 입력이 모든 체인에 전달, batch와 함께 가능	독립적인 호출이 여럿(검색+대화 요약)

모든 Runnable이 이 메서드를 자동으로 제공(내가 구현하지 않아도 따라옴).

> 파이프 타입(슬라이드): `retriever` (str→list[Document]) | `format_docs` (list[Document]→str) | `prompt` (dict→PromptValue) | `llm` (PromptValue→AIMessage) | `StrOutputParser` (AIMessage→str). 앞 출력 타입 = 뒤 입력 타입이어야 합성이 성립.
> 어긋나는 예: `retriever` → `llm` 직결 = `list[Document]` ≠ `PromptValue`. 문서 목록을 모델에 바로 넣을 수 없으므로 사이에 변환 단계(문서→문자열→프롬프트)가 반드시 필요.

- 재시도·폴백: `llm.with_retry(stop_after_attempt=3)` (일시 오류 재시도) / `.with_fallbacks([backup_llm])` (끝내 실패하면 다른 모델) / `.with_config(run_name=...)` (추적 이름표). 핵심: `with_*`의 반환값도 Runnable이라 합성 규칙이 안 깨져 체인의 나머지는 그대로 동작(본체 코드를 안 건드림).
- LangGraph = 분기·루프·상태가 필요할 때(선형 체인으로 안 되는 에이전트 흐름), LangSmith = 관측·추적(어떤 프롬프트가 들어가고 무엇이 검색됐는지), langchain-core/community = 패키지 분리.

서술형 대비 · 인과관계 한 줄 모음

질문	한 줄 답
글이 2배면 비용이 왜 4배인가?	어텐션이 모든 토큰 쌍을 비교해 계산량이 n^2이므로.
왜 문서를 통째로 넣지 않고 RAG를 쓰나?	컨텍스트 한도+길수록 비용·지연↑+불필요한 정보 간섭·가운데 정보 무시 → 필요한 조각만 골라 넣는 것이 성능이자 비용.
사전학습 모델은 왜 질문에 답하지 못하나?	다음 토큰 예측만 학습해 이어 쓰기만 하기 때문, SFT가 응답 형식을 가르침.
양자화하면 왜 성능이 떨어지나?	표현 가능한 값이 줄어 서로 다른 값이 같은 값으로 뭉쳐서.
한국어가 토큰을 더 쓰는 이유?	영어 데이터 비중이 커 병합 기회가 적고, 조사·어미로 같은 어간이 흩어져 빈도가 분산되어서.
FastAPI(비동기)가 LLM 서버에 적합한 이유?	LLM 호출은 대부분 응답 대기라, 대기 중 워커를 놓아 다른 요청을 처리하면 동시 처리량이 크게 늘어서.
URI에 동사를 넣지 않는 이유?	동작은 메서드가 표현하고 URI는 자원(명사)만 식별해 일관된 설계가 되도록.
구조화 출력은 언제 필요한가?	LLM 출력을 사람이 아니라 코드(백엔드)가 바로 파싱해 쓸 때(도구 호출 명령 등).
컨텍스트에 많이 넣을수록 좋지 않은 이유?	토큰 비용·지연 증가(n^2), 불필요한 정보의 간섭, 가운데 정보 무시로 오히려 성능↓ → 필요한 것만 선별.
하네스에서 검증 게이트가 왜 중요한가?	출력이 확률적이라 형식·금지 내용·실행 통과를 확인하지 않으면 틀린 결과가 다음 단계로 전파·누적되고, 루프의 복구 근거도 없어서.
루프에서 종료 조건이 가장 중요한 이유?	없거나 잘못되면 무한 재시도·비용 폭증 같은 안티패턴이 생겨 오류가 증폭되므로(횟수·예산 상한).
복구 단계에서 실패 원인을 입력에 넣는 이유?	같은 입력으로 재시도하면 같은 실패가 반복되기 때문.
"위험한 건 하지 마"라고 프롬프트에 써도 위험한 작업이 실행되는 이유?	자연어 지시는 보장이 아니므로 권한·승인을 코드 수준에서 막아야 함.
하네스가 먼저이고 루프가 그 위인 이유?	루프는 하네스의 실행을 반복하므로 하네스(도구·검증)가 부실하면 반복할수록 약점이 증폭되기 때문.
색인과 질의의 임베딩 모델이 같아야 하는 이유?	모델마다 벡터 공간이 달라 다르면 유사도 비교가 무의미해서(LLM 모델과는 달라도 됨: 원문 텍스트가 넘어감).
벡터 검색만으로 부족한 이유와 해결?	코드·고유명사·숫자(KCC-3 vs 5)는 벡터가 거의 같아 구분 못 함 → BM25(글자 일치)를 RRF로 합친 하이브리드.
RRF에서 점수 대신 순위를 쓰는 이유?	코사인(0~1)과 BM25(상한 없음)의 단위가 달라 점수를 그대로 합칠 수 없어서.
리랭킹의 장단점? 왜 상위 후보에만?	질문+조각을 함께 어텐션해 더 정밀하지만 후보마다 모델을 돌려 느리므로 1차 상위 몇십 개에만 적용.
정답이 후보엔 있는데 5위 밖일 때?	"잘못 고름" → 리랭킹·k 조정(청킹 문제가 아님).
고정 길이 청킹의 문제와 대응?	문장이 중간에 잘려 정답이 두 조각에 걸림 → 오버랩, 또는 구조·재귀 분할.
오버랩의 목적과 대가?	경계 근처 문장이 한 조각에는 온전히 들어가게 하는 보험, 대신 조각 수·임베딩·저장 비용 증가와 k 중복 점유.
규정 문서에 조 단위 청킹이 좋은 이유?	조항 하나가 질문 단위와 일치하고 한 주제+제목을 포함해 문맥이 보존되므로.
B+Tree를 벡터에 못 쓰는 이유?	벡터는 정렬이 안 되어 "어느 쪽으로 갈지" 정할 수 없어서 → ANN.
ANN에서 recall 1.0을 포기하는 이유?	마지막 몇 %를 위해 속도가 수십 배 느려지고, RAG는 하나만 맞아도 되며 리랭킹이 누락을 흡수하므로(목표 0.95~0.99).
IVF nlist를 키우면?	구역이 작아져 빠르지만 경계가 늘어 정답을 놓칠 확률↑. nprobe를 키우면 recall↑ 속도↓.

HNSW가 삭제에 불리한 이유?	삭제하면 그래프가 끊겨 도달 불가능한 노드가 생겨 삭제 표시 후 주기적 재구축이 필요.
PQ가 검색을 빠르게 하는 이유?	벡터를 대표 번호로 압축하고 질문당 거리표를 만들어 거리 계산을 표 조회+덧셈으로 바꿔서(단점은 정밀도 손실→재정렬).
후처리 필터의 문제?	검색 후 거르면 상위 k 중 조건에 맞는 것만 남아 k개를 못 채움 → 필터 인식 검색은 항상 k개.
Self-RAG와 CRAG의 차이?	Self-RAG는 생성 모델을 학습시켜 스스로 판단, CRAG는 검색 결과를 판단하는 별도 평가기가 있음.
GraphRAG를 모든 서비스에 안 쓰는 이유?	색인 비용이 매우 크고(LLM으로 모든 문서 훑기) 단순 사실 질문엔 이득이 없어서.
LangChain에서 Runnable 규격의 이점?	모든 단계가 같은 모양이라 이어 붙이기·감싸기(재시도·로깅)·바꿔 끼우기가 본체 코드 수정 없이 가능.
retriever를 llm에 바로 못 잇는 이유?	출력(list[Document])과 입력(PromptValue)의 형태가 달라 사이에 변환 단계가 필요.

상황 → 선택 (한 장 정리)

상황	선택	상황	선택
코드·고유명사 질의가 안 됨	하이브리드(BM25)	정답은 후보에 있는데 순위가 낮음	리랭킹
짧고 모호한 대화형 질문	질의 재작성	질문체≠문서체	HyDE
관계·전체 조망 질문	GraphRAG	쉬운/어려운 질문이 섞여 비용 절감	Adaptive
다단계 추론·리서치	Agentic RAG	표·이미지가 핵심	Multimodal
조각이 문맥을 잃음	Contextual Retrieval	관련 과거 대화만 꺼냄	선별 검색
최근 N턴만 기억	슬라이딩 윈도우	옛 대화는 요약	요약
외부 전송 금지 데이터	로컬 모델	빨리 시작·성능 우선	API 모델
수만 개 이하 조각	정확 탐색	수억 개·메모리 부족	IVF-PQ
정밀도 우선·일반적	HNSW	이미 PostgreSQL·수백만 이하·JOIN	pgvector
수천만+·복잡한 필터	전용 벡터 DB	권한 필터로 k개를 못 채움	필터 인식 검색
엉뚱한 도구 호출	도구 설명(레지스트리) 개선	위험한 작업이 실행됨	권한·승인(코드 수준)
오류가 계속 전파	검증 게이트	세션마다 처음부터	상태를 파일로 저장
도구가 수백 개	벡터 검색·라우터로 선별	무한 반복·비용 폭증	횟수·예산 상한
모델 능력 자체가 부족	모델 교체	검증 없이 끝냄·도구 오선택	하네스 개선
일시 오류 재시도 / 끝내 실패	with_retry / with_fallbacks	분기·루프·상태 필요	LangGraph

계산 모음

유형	공식 / 예
모델 메모리	파라미터 수 × 바이트. 13B FP16=26GB, 70B INT4=35GB, 7B INT8=7GB
처리량(동기)	워커 수 ÷ 응답 시간. 4÷5초=0.8건/초
어텐션	길이 n배 → 계산 n^2배 (2배→4배)
오류 누적	$0.9^{10} \approx 0.35$ (단계별 90%, 10단계)
BPE 어휘	초기 글자 수 + 병합 횟수
임베딩 저장 용량	조각 수 × 차원 × 4B. 100만×1024×4B ≈ 4GB, 100만×768×4B ≈ 3GB
RRF	Σ 1/(60+순위). 벡터 2위·BM25 1위=1/62+1/61, 양쪽 1위=2/61≈0.0328
지표	Precision@5 = 관련 수/5(3개→0.6) · MRR = 1/첫 정답 순위(3위→1/3) · Recall@5 = 정답이 상위 5에 든 질문 수/전체(18/20=0.9)
오버랩	보폭 = 크기−오버랩(500/100→400 시작). 조각 수 = 전체 길이/보폭 (10만/450≈223)
IVF 비교 횟수	nlist + nprobe×(N/nlist). 100만, 1,000, 10 → 1,000+10×1,000=11,000 (≈1%)
ANN Recall	정확 탐색 상위 10 중 9개 찾음 = 0.9
PQ 압축률	(차원×4B)÷(조각 수×1B). 1024차원·8조각 → 4096÷8 = 512배

정리 기준: 수업 슬라이드 원문과 10/13·10/15 복습 녹음. 6주차 LangGraph 상세와 CRAG·Agentic RAG 도식은 슬라이드가 이미지라 녹음 설명을 따랐음.